

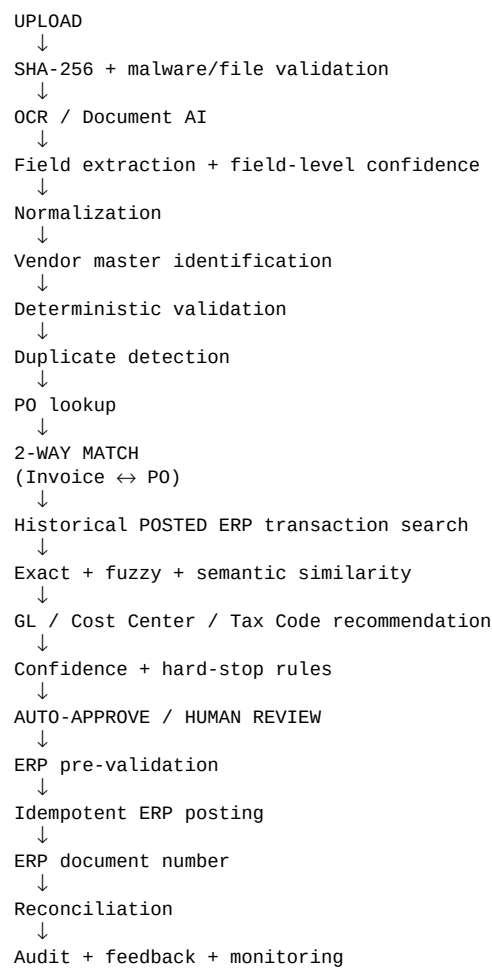
AI Accounts Payable / ERP Automation

Production V2 — Real-Time Scenarios & Exception Handling

Purpose: extend the earlier production blueprint into a realistic AP automation design covering 2-way matching, historical ERP similarity, GST/TDS, duplicates, credit notes, approvals, ERP failures, reconciliation, security, monitoring and operational edge cases.

Important: This is an implementation blueprint with representative TypeScript/SQL code. Actual SAP/Oracle/D365 API payloads, tax rules, approval hierarchy and accounting policy must be configured for the target organization.

1. Complete Production Workflow



The AI score is a decision-support signal. It must never bypass deterministic accounting, tax, duplicate, authorization or ERP validation rules.

2. Scenario Matrix — PO and 2-Way Matching

Scenario	System behavior	Decision
PO fully matches	Vendor, PO, currency and invoice amount/line values match within configured tolerance.	Eligible for approval
Price variance	Invoice price > PO price beyond tolerance.	Exception
Quantity variance	Invoice quantity > available PO quantity.	Exception
Partial invoice	Invoice consumes only part of PO quantity/value.	Allow; update remaining PO balance
Multiple invoices against PO	Track cumulative invoiced amount/quantity.	Block when remaining balance is exceeded
PO closed/cancelled	ERP status check fails.	Block
No PO	Route to configured non-PO workflow.	Approval required
Multiple possible POs	Rank candidates using vendor, currency, amount, date and description.	Review if ambiguous
PO changed after receipt	Compare current ERP PO and relevant historical version if available.	Exception/review

3. 2-Way Matching Service

```
export function twoWayMatch(invoice, po) {
  const priceOk =
    Math.abs(invoice.subtotal - po.expectedSubtotal) <=
      po.toleranceAmount;

  const qtyOk = invoice.lines.every(line => {
    const poLine = po.lines.find(x => x.poLine === line.poLine);
    return poLine &&
      line.quantity <= poLine.remainingQuantity &&
      Math.abs(line.unitPrice - poLine.unitPrice) <= poLine.priceTolerance;
  });

  const vendorOk = invoice.vendorId === po.vendorId;
```

```

const currencyOk = invoice.currency === po.currency;
const statusOk = po.status === "OPEN";

return {
  passed: vendorOk && currencyOk && statusOk && priceOk && qtyOk,
  vendorOk, currencyOk, statusOk, priceOk, qtyOk
};
}

```

Do not silently convert a failed match into a pass because historical invoices look similar. Historical similarity and PO validation are separate evidence streams.

4. Duplicate Detection — More Than File Hash

```

async function detectDuplicate(inv) {
  const exact = await db.invoice.findFirst({
    where: {
      vendorId: inv.vendorId,
      normalizedInvoiceNo: inv.normalizedInvoiceNo,
      fiscalYear: inv.fiscalYear,
      status: { in: ["POSTED", "APPROVED", "PENDING_REVIEW"] }
    }
  });

  if (exact) return { duplicate: true, reason: "VENDOR_INVOICE_FY" };

  const near = await findHistoricalCandidates({
    vendorId: inv.vendorId,
    amount: inv.totalAmount,
    date: inv.invoiceDate,
    days: 90
  });

  return {
    duplicate: near.some(x =>
      x.amount === inv.totalAmount &&
      similarity(x.invoiceNumber, inv.invoiceNumber) >= 0.90
    ),
    candidates: near
  };
}

```

- Check vendor + invoice number + fiscal year, not invoice number alone.
- Also check credit/debit note relationships, reversed documents and prior ERP posting IDs.
- Keep file SHA-256 for exact-file duplicate detection.

5. Credit Notes / Debit Notes / Advances

```

type DocumentType =
  | "STANDARD_INVOICE"
  | "CREDIT_NOTE"
  | "DEBIT_NOTE"
  | "ADVANCE"
  | "PROFORMA";

function validateDocumentType(inv) {
  if (inv.documentType === "PROFORMA") {
    return { allowedToPost: false, reason: "PROFORMA_NOT_POSTABLE" };
  }

  if (inv.documentType === "CREDIT_NOTE" && !inv.referenceInvoiceNo) {
    return { allowedToPost: false, reason: "CREDIT_NOTE_REFERENCE_REQUIRED" };
  }

  if (inv.documentType === "ADVANCE") {
    return { allowedToPost: false, reason: "ADVANCE_WORKFLOW_REQUIRED" };
  }

  return { allowedToPost: true };
}

```

Credit notes should normally be linked to the original invoice or an organization-approved adjustment workflow. Proforma documents should not be treated as ordinary payable invoices.

6. India GST Validation

```

function validateGST(inv, vendor, company) {

```

```

const errors = [];

if (!isValidGSTIN(vendor.gstin))
  errors.push("INVALID_VENDOR_GSTIN");

if (inv.taxableAmount < 0)
  errors.push("INVALID_TAXABLE_AMOUNT");

const expectedTax = calculateGST({
  taxableAmount: inv.taxableAmount,
  supplyType: determineSupplyType(company.gstin, vendor.gstin),
  rate: inv.gstRate
});

if (!withinTolerance(expectedTax.total, inv.gstTotal, 1.00))
  errors.push("GST_AMOUNT_MISMATCH");

if (inv.taxCode !== expectedTax.taxCode)
  errors.push("TAX_CODE_MISMATCH");

return errors;
}

```

- Support CGST + SGST for intra-state supplies and IGST for inter-state supplies according to configured tax rules.
- Validate GSTIN, tax rate, taxable value, tax amounts and tax code.
- For production, use an authoritative tax-rule source and version the rules by effective date.
- Add organization-specific e-invoice/IRN and e-way-bill validation where applicable.

7. TDS Validation

```

function evaluateTDS(ctx) {
  const rule = findTDSRule({
    vendorType: ctx.vendorType,
    serviceType: ctx.serviceType,
    section: ctx.tdsSection,
    effectiveDate: ctx.invoiceDate
  });

  if (!rule) return { applicable: false };

  const thresholdReached =
    ctx.priorFinancialYearBase + ctx.currentBase >= rule.threshold;

  return {
    applicable: thresholdReached,
    section: rule.section,
    rate: ctx.vendorHasValidCertificate
      ? rule.certificateRate ?? rule.rate
      : rule.rate,
    expectedTDS: thresholdReached
      ? round(ctx.tdsBase * rule.rate / 100, 2)
      : 0
  };
}

```

TDS sections, thresholds, rates and exceptions must be maintained as effective-dated configuration. Do not hard-code tax rates in business logic.

8. Accounting Recommendation — GL / Cost Center / Tax Code

```

async function recommendAccounting(inv, history) {
  const candidates = await retrieveSimilarPostedTransactions(inv);

  const scored = candidates.map(x => ({
    ...x,
    score:
      0.30 * vendorSimilarity(inv, x) +
      0.25 * descriptionSimilarity(inv, x) +
      0.20 * amountSimilarity(inv, x) +
      0.15 * taxSimilarity(inv, x) +
      0.10 * costCenterSimilarity(inv, x)
  })).sort((a,b) => b.score - a.score);

  const top = scored[0];

  return {
    gl: top?.gl,

```

```

    costCenter: top?.costCenter,
    taxCode: top?.taxCode,
    evidence: scored.slice(0, 5),
    confidence: top?.score ?? 0
  };
}

```

Historical data must be treated as evidence, not truth. If historical postings are inconsistent, the system should lower confidence and route the invoice to review.

9. New Vendor / New GL / New Cost Center

- No vendor history → use vendor master attributes, PO context and accounting policy; do not pretend historical confidence exists.
- No historical GL match → use deterministic mapping rules or human approval.
- New cost center → require master-data validation before posting.
- Multiple equally valid GLs → do not auto-select solely from a weak similarity score.

10. Confidence Engine + Hard Stops

```

function decide(ctx) {
  const hardStops = [
    ctx.duplicate,
    !ctx.vendorValid,
    !ctx.taxValid,
    !ctx.erpMasterDataValid,
    ctx.twoWayMatch.passed === false,
    ctx.documentTypeBlocked,
    ctx.postingPeriodClosed
  ];

  if (hardStops.some(Boolean)) {
    return { decision: "BLOCK_OR_REVIEW", confidence: 0 };
  }

  if (ctx.confidence >= 0.95 &&
    ctx.fieldConfidence >= 0.95 &&
    ctx.recommendationConfidence >= 0.95) {
    return { decision: "ELIGIBLE_FOR_AUTO_APPROVAL",
      confidence: ctx.confidence };
  }

  return { decision: "HUMAN_REVIEW", confidence: ctx.confidence };
}

```

The 95% threshold should be configurable and calibrated using labeled production outcomes. A mathematical score of 95% does not mean 95% real-world accuracy unless calibration and monitoring support that interpretation.

11. Approval Matrix

```

function requiredApprover(invoice, policy) {
  if (invoice.sodConflict) return "BLOCK";

  if (invoice.totalAmount >= policy.level3.amount)
    return "LEVEL_3";

  if (invoice.totalAmount >= policy.level2.amount)
    return "LEVEL_2";

  return "LEVEL_1";
}

```

- Maker-checker: uploader/processor must not approve their own transaction.
- Support amount, company code, cost center, vendor and exception-based routing.
- Support delegation, unavailable approvers, rejection, resubmission and SLA escalation.
- Keep every approval/rejection as an immutable audit event.

12. ERP Failure, Timeout and Retry

```

async function postWithIdempotency(invoice) {

```

```

const key = `INV:${invoice.id}`;

const existing = await db.erpPosting.findUnique({
  where: { idempotencyKey: key }
});

if (existing?.status === "POSTED")
  return existing;

const result = await erp.postInvoice({
  ...invoice,
  idempotencyKey: key
});

await db.erpPosting.upsert({
  where: { idempotencyKey: key },
  create: { idempotencyKey: key, ...result },
  update: { ...result }
});

return result;
}

```

- ERP timeout → retry using the same idempotency key.
- Unknown outcome → query ERP by external reference before retrying.
- Validation error → stop and show ERP error; do not blindly retry.
- ERP downtime → queue posting and alert operations.
- Crash after successful ERP post → reconciliation must recover the ERP document number.

13. Reconciliation

```

async function reconcile(posting) {
  const erpDoc = await erp.findByExternalReference(
    posting.idempotencyKey
  );

  if (!erpDoc) {
    return markReconciliation("MISSING_IN_ERP");
  }

  if (!amountsMatch(posting.invoice, erpDoc)) {
    return markReconciliation("AMOUNT_MISMATCH");
  }

  return markReconciliation("MATCHED", {
    erpDocumentNo: erpDoc.documentNo
  });
}

```

14. Historical Posted Ledger + Hybrid Search

```

SELECT id, vendor_id, invoice_number, amount, gl_code,
       cost_center, tax_code, description,
       1 - (embedding <=> $1::vector) AS semantic_similarity
FROM posted_invoice_history
WHERE vendor_id = $2
ORDER BY embedding <=> $1::vector
LIMIT 20;

```

Use exact filters first where possible, then fuzzy text similarity and vector/semantic similarity. pgvector can support vector similarity search inside PostgreSQL; production indexing and query strategy should be benchmarked against your actual corpus.

15. File Security

- Allow only approved document types and enforce file-size limits.
- Validate extension, MIME type and file signature/magic bytes.
- Generate server-side storage names; never trust the original filename.
- Store uploaded files privately and expose them through authorization-controlled access.
- Run malware scanning and, where appropriate, content disarm/reconstruction.

- Log upload, extraction, review, approval and deletion events.

16. Multiple Files / Multi-Page / Batch Upload

```
async function processUpload(file) {
  const pages = await documentAI.splitPages(file);
  const documents = await documentAI.detectDocuments(pages);

  if (documents.length > 1) {
    return queueEachDocument(documents);
  }

  return queueInvoiceExtraction(documents[0]);
}
```

A production system should distinguish a multi-page invoice from a PDF containing several invoices. Each invoice should receive its own immutable processing ID.

17. Queue and Worker Design

```
invoiceQueue.add("process-invoice", { invoiceId }, {
  attempts: 5,
  backoff: { type: "exponential", delay: 2000 },
  removeOnComplete: 1000
});

// Worker
worker.on("failed", async (job, err) => {
  await audit.log("JOB_FAILED", {
    jobId: job.id,
    invoiceId: job.data.invoiceId,
    error: err.message
  });
});
```

- Use separate queues for extraction, matching, embeddings and ERP posting where workloads differ.
- Use dead-letter handling for permanently failed jobs.
- Make every worker step idempotent.
- Track processing latency and queue depth.

18. Production Data Model — Core Tables

```
Invoice
Vendor
PurchaseOrder
PurchaseOrderLine
InvoiceLine
PostedInvoiceHistory
AccountingRecommendation
Approval
ERPPosting
Reconciliation
AuditEvent
Feedback
ProcessingJob
DocumentFile

Key identifiers:
- invoice.id
- documentFile.sha256
- invoice.vendorId + normalizedInvoiceNo + fiscalYear
- erpPosting.idempotencyKey
- erpPosting.erpDocumentNo
```

19. Security / RBAC / SoD

```
const permissions = {
  AP_PROCESSOR: ["invoice.read", "invoice.review"],
  AP_APPROVER: ["invoice.read", "invoice.approve"],
  ERP_POSTER: ["invoice.read", "invoice.post"],
  AUDITOR: ["audit.read"],
  ADMIN: ["config.manage"]
};
```

Enforce authorization server-side. Do not rely on frontend buttons or hidden routes. Add OIDC/SSO, MFA through the identity provider, least privilege, segregation-of-duties checks and service-account controls.

20. Monitoring and Alerting

- OCR failure rate and extraction confidence distribution.
- Vendor-match confidence and false-match rate.
- Duplicate detection rate and confirmed duplicate rate.
- 2-way match exception rate.
- Human-review rate and approval SLA.
- ERP posting success, retry, timeout and reconciliation rates.
- AI recommendation accuracy by vendor, document type and accounting category.
- Queue depth, worker failures and dead-letter count.
- Model/rule version on every decision.

21. Feedback Loop

```
await db.feedback.create({
  data: {
    invoiceId,
    predictedGl,
    actualGl: reviewer.gl,
    predictedCostCenter,
    actualCostCenter: reviewer.costCenter,
    predictedTaxCode,
    actualTaxCode: reviewer.taxCode,
    reason: reviewer.changeReason,
    modelVersion
  }
});
```

Use reviewed outcomes for evaluation and controlled model improvement. Never silently retrain on unverified accounting changes.

22. Important Edge Cases Checklist

- Foreign currency invoices and exchange-rate date selection.
- Round-off differences and configurable monetary tolerance.
- Negative/zero invoices.
- Tax-inclusive vs tax-exclusive pricing.
- Withholding/TDS deductions affecting payable amount.
- Blocked vendor, inactive vendor or vendor bank-account changes.
- Closed fiscal period / invalid posting date.
- Intercompany transactions.
- Capex vs Opex and asset-account workflow.
- Prepaid expenses and accruals.
- Recurring invoices and contract-based invoices.
- Invoices with missing PO numbers.
- Handwritten/poor-quality documents.
- Mixed-language documents.
- Invoice number formatting variations.
- Vendor name changes and aliases.

- ERP master-data synchronization failures.

23. API Endpoints

```
POST /api/invoices/upload
GET /api/invoices/:id
GET /api/invoices/:id/evidence
POST /api/invoices/:id/review
POST /api/invoices/:id/approve
POST /api/invoices/:id/reject
POST /api/invoices/:id/post
GET /api/invoices/:id/reconciliation
GET /api/audit/:invoiceId
POST /api/feedback
GET /api/dashboard/metrics
GET /api/exceptions
```

24. Recommended Status Machine

```
RECEIVED
→ EXTRACTING
→ VALIDATING
→ MATCHING
→ RECOMMENDED
→ PENDING_APPROVAL
→ APPROVED
→ POSTING
→ POSTED
→ RECONCILED
```

```
Exception states:
EXTRACTION_FAILED
DUPLICATE_BLOCKED
VALIDATION_FAILED
MATCH_EXCEPTION
APPROVAL_REJECTED
ERP_VALIDATION_FAILED
ERP_TIMEOUT
POSTING_UNKNOWN
RECONCILIATION_FAILED
```

25. What Makes This Production-Ready

- PostgreSQL is the system of record rather than mutable in-memory arrays.
- OCR/AI, matching, approval and ERP posting are separate services/interfaces.
- 2-way matching is deterministic and configurable.
- Historical posted transactions provide evidence for accounting recommendations.
- AI confidence is combined with field confidence and hard-stop rules.
- ERP posting is idempotent and reconciled.
- Every important action is auditable.
- Exceptions are first-class workflows rather than hidden failures.
- Queue workers make long-running AI/OCR/ERP operations resilient.
- Security, RBAC, file controls, monitoring and testing are included.

26. Final Implementation Sequence

1. Create PostgreSQL schema and migrations.
2. Import/synchronize vendor, PO and posted ERP history.
3. Implement secure upload and document storage.
4. Integrate OCR/document AI and persist field-level confidence.
5. Implement normalization and vendor identification.
6. Implement duplicate detection.

7. Implement 2-way PO matching and tolerance rules.
8. Build historical retrieval + exact/fuzzy/semantic similarity.
9. Build GL/cost-center/tax recommendation.
10. Implement confidence calibration and hard stops.
11. Build review/approval UI and SoD checks.
12. Integrate ERP pre-validation.
13. Implement idempotent ERP posting.
14. Implement reconciliation and unknown-outcome recovery.
15. Add audit, metrics, alerts and feedback.
16. Run unit, integration, end-to-end, load and failure-injection tests.
17. Pilot with human approval before enabling any automatic posting.

27. Production Readiness Verdict

This V2 blueprint covers the major real-time AP scenarios discussed earlier, but it is still not a plug-and-play ERP implementation. The remaining organization-specific work is mainly the real OCR/Document AI provider, actual ERP APIs, tax-rule source, vendor/PO/master-data synchronization, identity provider, approval policy, accounting rules, infrastructure and production testing.

Recommended rollout: **shadow mode** → **human-approved pilot** → **controlled auto-approval for narrowly defined high-confidence cases** → **gradual expansion**. Do not start with unrestricted automatic ERP posting.